

Predicting Billboard #1 Hits

Authors: May Eindra Tet Toe, Anastasia Tountas, Tran Anh Thu Phung, Harry Nguyen

Summary

Insert a summary of your analysis here.

Introduction

Insert an introduction to your analysis here.

Methods and Results

Overview of our Methodology

The Billboard Hot 100 dataset provides us with two datasets: `billboard.csv`, which contains one row per song that reached #1 along with its features, and `topics.csv`, a reference list of lyrical theme labels (`lyrical_topics`). Our team decided to combine the two datasets to `billboard_clean` in order to create more comprehensive data to better equip our model. Here is an overview of our data analysis methodology:

1. **Reading & Wrangling:** read and wrangle the dataset into one tidy combined dataset
2. **Exploratory Data Analysis:** explore the data to understand the distribution of features and their relationships with the target variable
3. **Model Building:** build a regression model to predict the number of weeks a song stays at #1 based on its features
4. **Model Evaluation:** evaluate the performance of our model using appropriate metrics and validation techniques
5. **Results and Conclusion:** summarize our findings, discuss the implications of our results, and suggest potential future work or improvements to our analysis

Load libraries and set seed

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.6
## v forcats    1.0.1      v stringr   1.6.0
## v ggplot2    4.0.1      v tibble    3.3.0
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.2.0
```

```
## -- Conflicts ----- tidyverse_conflicts() --
```

```
## x dplyr::filter() masks stats::filter()
```

```
## x dplyr::lag()     masks stats::lag()
```

```
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(tidyuesdayR)
```

```
library(knitr)
```

```
library(dplyr)
```

```
set.seed(123)
```

Reading & Wrangling our dataset from the web into R

- The two datasets were joined using a `left_join()` on `lyrical_topic` (billboard) and `lyrical_topics`(topics).
- **Identifier columns** (song, artist, date) were removed as they are not predictive features.
- **non_consecutive** was removed to prevent data leakage — it can only be determined after a song has completed its full chart run.
- **rating_1, rating_2, rating_3** were removed in favour of **overall_rating** (their mean) to avoid multicollinearity.
- **Free-text columns** were removed avoid the high dimensionality and noise associated with Natural Language Processing: `lyrics`, `u_s_artwork`, `sound_effects`, `song_structure`, `featured_artists`, `songwriters`, `songwriters_w_o_interpolation_sample_credits`, `producers`, `double_a_side`, and `talent_contestant`.
- **High-cardinality columns** removed because they generalise poorly: `label`, `parent_label`, and `artist_place_of_origin`.
- **Redundant columns** were removed: `keys` is replaced by `simplified_key`; `cdr_style` and `discogs_style` are replaced by `cdr_genre` and `discogs_genre` respectively.
- **discogs_genre** was removed in favour of `cdr_genre` because `cdr` has more reliable class definitions, thus improving internal validity.
- **Ambiguous columns** were removed: `featured_in_a_then_contemporary_play`, `featured_in_a_then_contemporary_t_v_show`, and `featured_in_a_then_contemporary_t_v_show`.
- All remaining character columns were converted to factors using `as.factor()`, and rows with any missing values were dropped with `drop_na()`.

```
# Load the dataset
```

```
tuesdata <- tidyTuesdayR::tt_load(2025, week = 34)
```

```
## ---- Compiling #TidyTuesday Information for 2025-08-26 ----
```

```
## --- There are 2 files available ---
```

```
##
```

```
##
```

```
## -- Downloading files -----
```

```
##
```

```
## 1 of 2: "billboard.csv"
```

```
## 2 of 2: "topics.csv"
```

```
# Extract the datasets
```

```
billboard <- tuesdata$billboard
```

```
topics <- tuesdata$topics
```

```
# Join the two datasets
```

```
billboard_joined <- billboard |>
```

```
  left_join(topics, by = c("lyrical_topic" = "lyrical_topics"))
```

```
billboard_clean <- billboard_joined |>
```

```
  # Remove identifier columns
```

```
  select(-c(song, artist, date)) |>
```

```
  # Remove data-leakage column
```

```
  select(-non_consecutive) |>
```

```
  # Remove individual judge scores to avoid multicollinearity
```

```
  select(-c(rating_1, rating_2, rating_3)) |>
```

```
  # Remove free-text columns
```

```
  select(-c(
    lyrics,
```

```

u_s_artwork,
sound_effects,
song_structure,
featured_artists,
songwriters,
songwriters_w_o_interpolation_sample_credits,
producers,
double_a_side,
talent_contestant
)) |>
# Remove high-cardinality columns
select(-c(label, parent_label, artist_place_of_origin)) |>
# Remove redundant columns and discogs_genre
select(-c(cdr_style, discogs_style, keys, discogs_genre)) |>
# Remove ambiguous columns
select(-c(
  featured_in_a_then_contemporary_play,
  featured_in_a_then_contemporary_film,
  featured_in_a_then_contemporary_t_v_show
)) |>
# Convert remaining character columns to factors
mutate(across(where(is.character), as.factor)) |>
drop_na()

```

Exploratory Data Analysis

We first examine the distribution of our target variable, `weeks_at_number_one`, to check for skewness.

```

ggplot(billboard_clean, aes(x = weeks_at_number_one)) +
  geom_histogram(binwidth = 1, fill = "steelblue", colour = "white") +
  labs(
    title = "Distribution of Weeks at Number One",
    x = "Weeks at Number One",
    y = "Count"
  ) +
  theme_minimal()

```

The distribution is heavily right-skewed. The majority of songs spent only 1–3 weeks at #1, with counts dropping off sharply afterwards. A small number of songs stayed for 10 or more weeks, forming a long right tail.

Next, we examine the distributions of the five Spotify audio features, to identify any skewness that may require transformation before modelling.

```

billboard_clean |>
  select(bpm, energy, danceability, happiness, acousticness) |>
  pivot_longer(
    cols = everything(),
    names_to = "feature",
    values_to = "value"
  ) |>
  ggplot(aes(x = value)) +
  geom_histogram(fill = "steelblue", colour = "white", bins = 30) +
  facet_wrap(~feature, scales = "free") +
  labs(

```

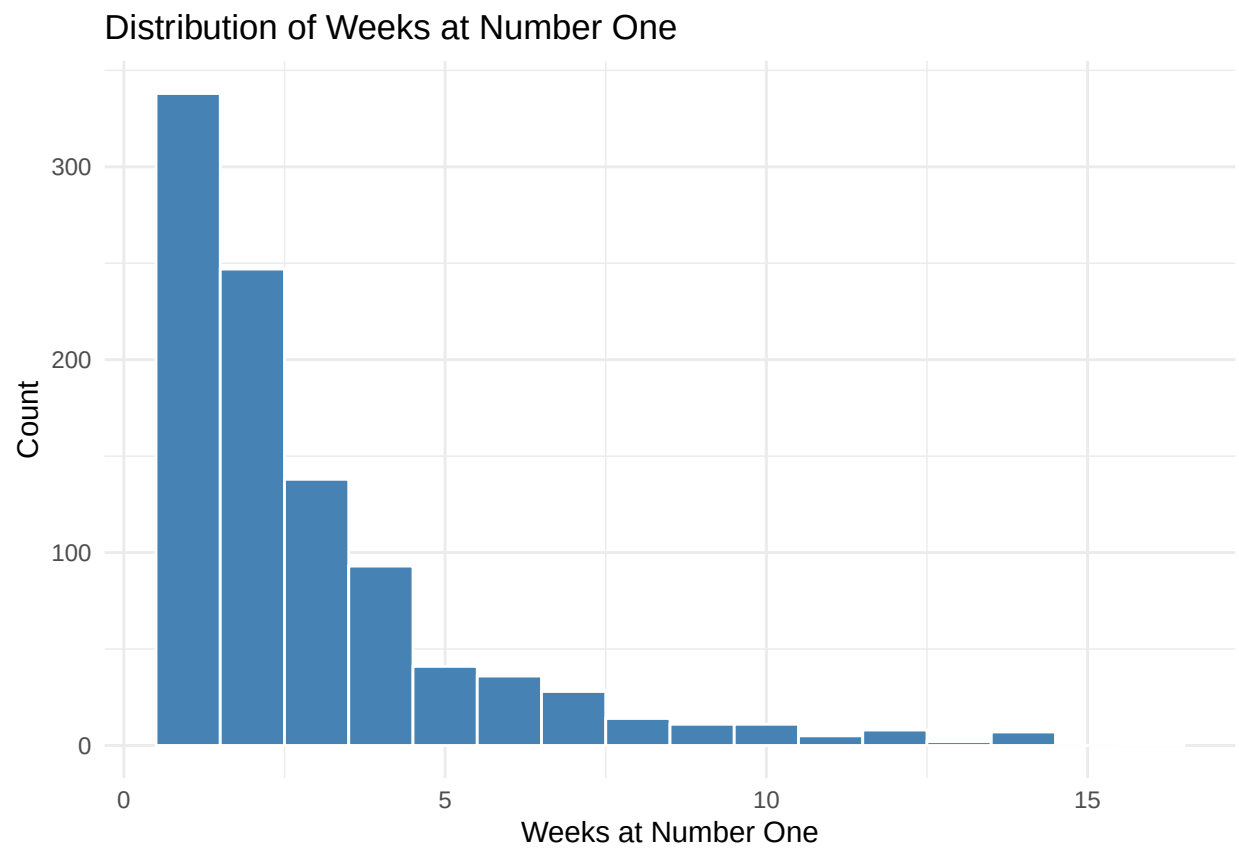


Figure 1: Distribution of weeks at number one.

```

title = "Distributions of Spotify Audio Features",
x = "Value",
y = "Count"
) +
theme_minimal()

```

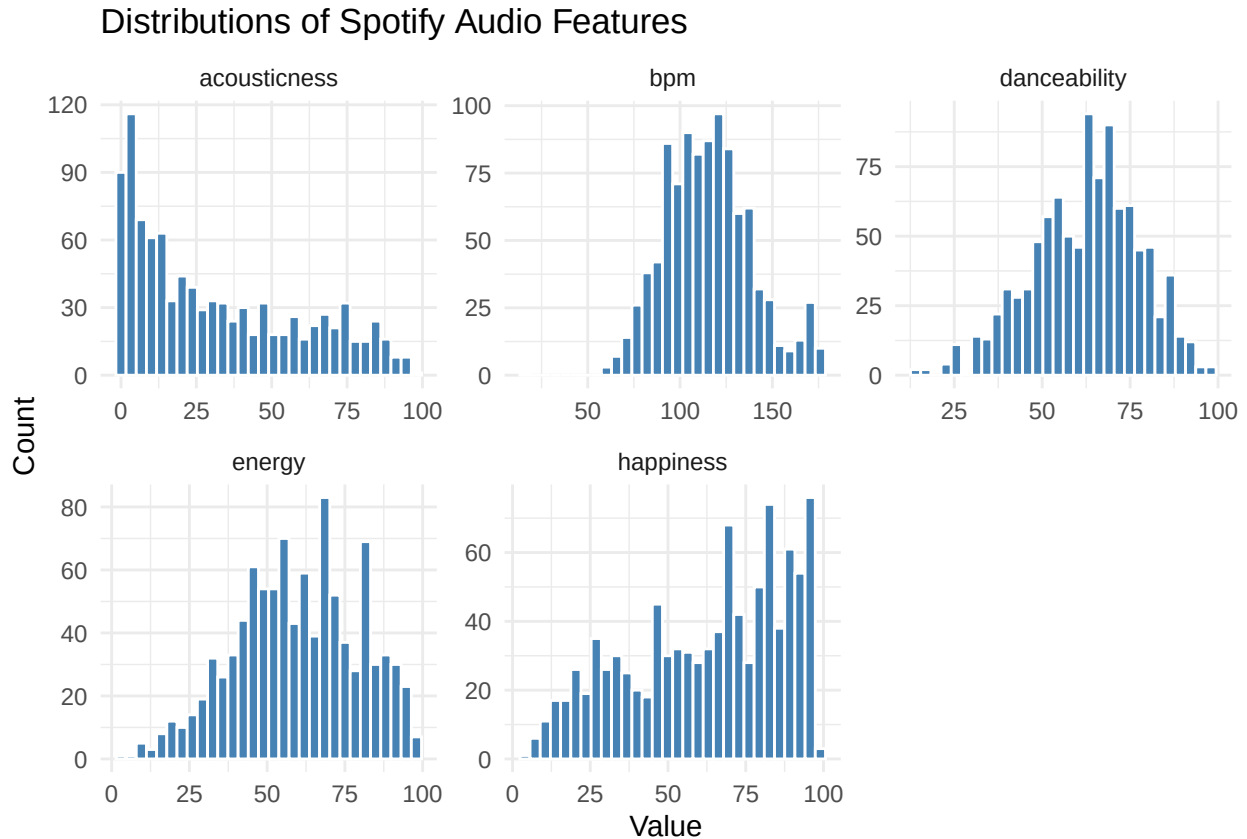


Figure 2: Distributions of the five Spotify audio features.

acousticness is heavily right-skewed, with most songs clustered near zero. **happiness** shows a bimodal pattern with peaks around 30 and around 80. **energy** is very slightly left-skewed, while **bpm** and **danceability** are roughly normal.

Finally, we compare average **weeks_at_number_one** by genre to see whether certain genres are associated with longer chart runs.

```

billboard_clean |>
  group_by(cdr_genre) |>
  summarise(mean_weeks = mean(weeks_at_number_one), .groups = "drop") |>
  ggplot(aes(
    x = reorder(cdr_genre, mean_weeks),
    y = mean_weeks
  )) +
  geom_col(fill = "steelblue") +
  coord_flip() +
  labs(
    title = "Average Weeks at Number One by Genre",

```

```
x = "Genre",
y = "Average Weeks at Number One"
) +
theme_minimal()
```

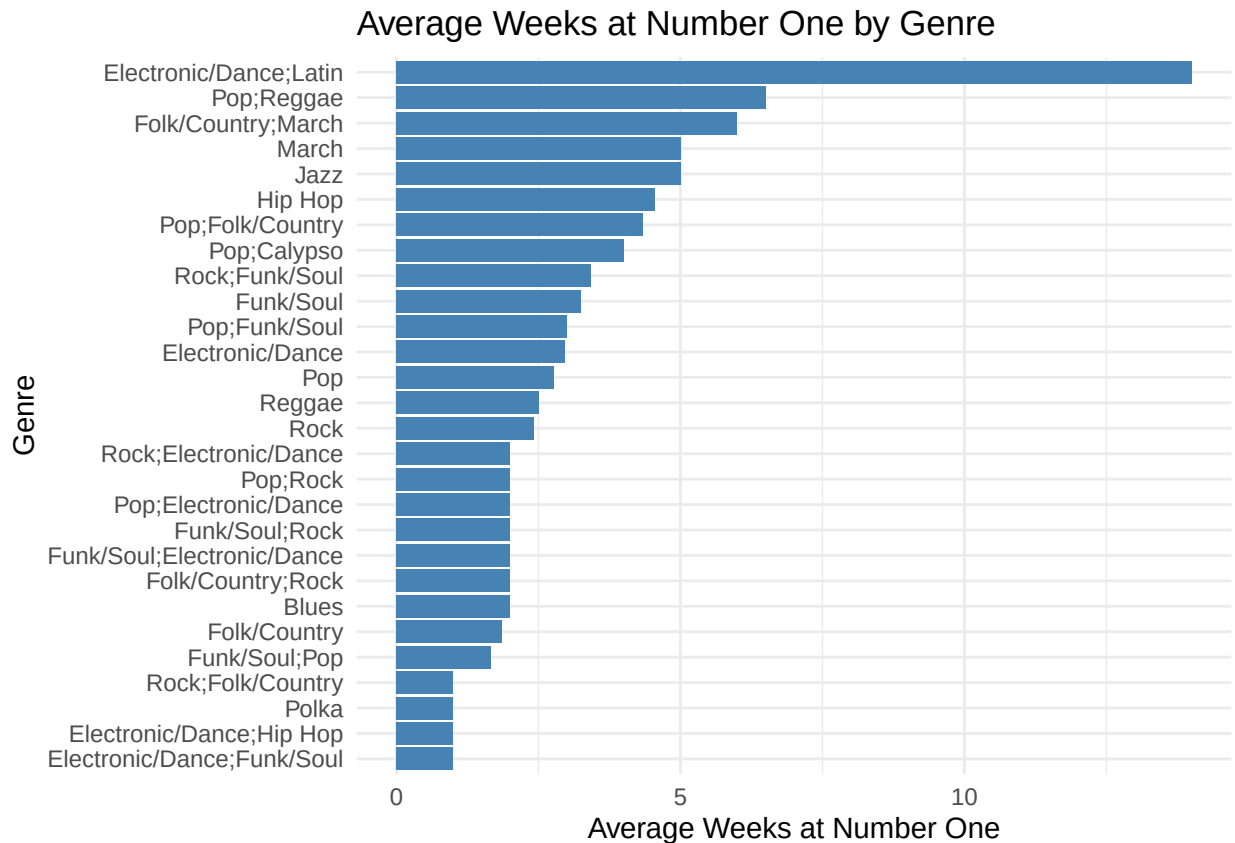


Figure 3: Average weeks at number one by genre.

Electronic/Dance;Latin seems to have the highest average (~10 weeks). Pop;Reggae and Folk/Country;March also rank highly. Mainstream genres like Pop and Rock sit in the middle of the range. At the lower end, we can find niche genres like Electronic/Dance;Funk/Soul, averaging to 1 week.

Feature Transformations

Based on the Exploratory Data Analysis, three variables require transformation before modelling:

- **weeks_at_number_one** is heavily right-skewed
- **acousticness** is also heavily right-skewed
- **happiness** is mildly left-skewed

Therefore, we apply log transformation to these three variables to compresses the tail and produce a more symmetric target distribution. We will be using `log1p()` instead of `log()` to handle any zero values in the data.